TDD gyakorlati feladat – Szökőév függvény

Ne feledd! **Red** 🔴 **-> Green** 🟢 **-> Refactor** ✨

Feladat – Szökőév függvény megvalósítása

A feladatod egy olyan függvény (IsLeapYear) elkészítése, amely egy évszámot (egész számot) kap paraméterül, és egy logikai értéket (bool) ad vissza az alábbi szabályok szerint:

- **Alapszabály:** Ha az évszám osztható 4-gyel, akkor az év **szökőév** (true).
- **Kivétel:** Ha az évszám osztható 100-zal, akkor **nem szökőév** (false).
- **Kivétel a kivétel alól:** Ha az évszám osztható 400-zal is, akkor **mégis szökőév** (true).
- **Minden más esetben: Nem szökőév** (false).

A munkafolyamat: szigorúan TDD!
Ne írd meg a kész algoritmust egyszerre! Kövesd az alábbi iterációkat:

1. **Iteráció (Az alap eset):** Írj egy tesztet egy sima évre (pl. 2021), ami elvárja, hogy a válasz false legyen. Futtasd le (Piros 🔴). Írd meg a legminimálisabb kódot (pl. return false;), hogy átmenjen (Zöld 🟢).

2. **Iteráció (A 4-es szabály):** Írj tesztet a 2024-es évre. Bukjon el! Módosítsd a kódot, hogy oszthatóság esetén true-t adjon.

3. **Iteráció (A 100-as kivétel):** Ismételd meg a folyamatot az 1900-as évre. Ennek el kell buknia, hiszen a kódod most true-t adna rá, de a szabály szerint ez false. Javítsd a kódot!

4. **Iteráció (A 400-as szabály):** Írj tesztet a 2000-es évre. Javítsd a kódot, hogy ez az év újra szökőév (true) legyen.

A **felhasznált eszközök, technológiák** ezek legyenek: Rider, C#, NUnit nuGet package.

Mit adj le?

A feladat teljesítéséhez egy ZIP fájlt tölts fel, ami tartalmazza:

1. **Képernyőkép 1:** Egy olyan állapot, amikor a teszted éppen **megbukik** (Red fázis).
2. **Képernyőkép 2:** Amikor a teszted már **sikeres** (Green fázis).
3. **A kész kód:** A forráskódod és a hozzá tartozó tesztfájl.
4. **Refaktorálás:** Írj egy mondatot: változtattál-e valamit a kódon a végén, hogy szebb és/vagy olvashatóbb legyen, miután már minden teszted zöld volt? (A mondatot a forráskódba is írhatod, kommentezve.)